Progetto di Elementi di Intelligenza Artificiale

Esplorazione dei labirinti

A.A.: 2025/26

Di Costanzo Mariagrazia (N46007023)

Incognito Vincenzo (N46007049)

De Crescenzo Davide (N46007180)

Indice

1. ***Obiettivo del progetto***
2. ***Dataset***
3. ***Gli algoritmi di ricerca***
4. ***Confronto tra algoritmi***

Obiettivo del progetto

L'obiettivo del progetto è quello di risolvere dei labirinti applicando diversi algoritmi di ricerca e confrontandone i risultati. I labirinti sono modellati su griglie bidimensionali, rappresentati in matrici di numeri interi, $n * n$, dove ogni valore corrisponde ad un tipo di cella percorribile, o meno, identificata da un colore specifico.

0: "Percorso",

1: "Muro",

2: "Inizio",

3: "Fine",

4: "Melma",

5: "MuroIllusione",

6: "FruttoDimenticanza",

7: "PercorsoTrovato",

8: "PercorsoHillClimbing",

9: "PercorsoDLS",

10: "PercorsoA*",

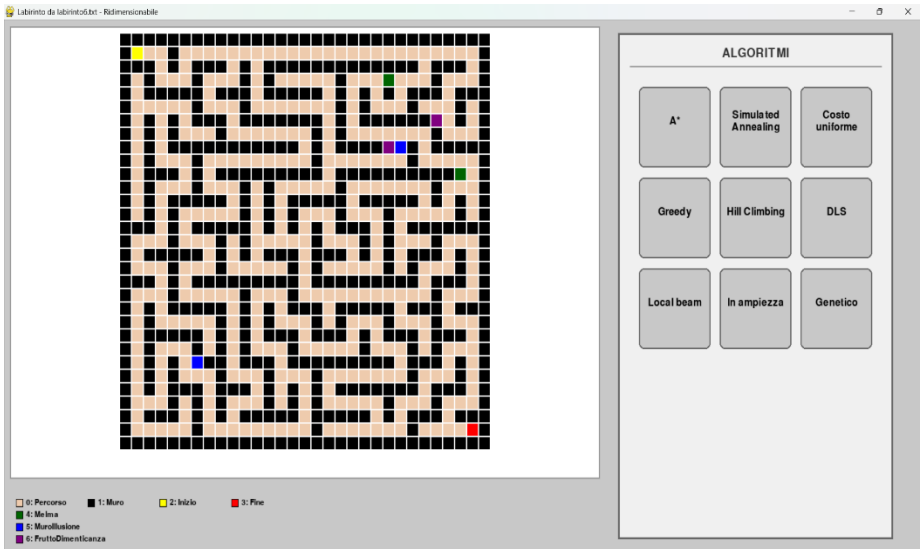
11: "PercorsoUCS",

12: "PercorsoSA",

13: "Local Beam",

14: "Ampiezza",

15: "Genetici".



Sulla destra del labirinto si presenta un pannello con una griglia di pulsanti, ciascuno associato ad un algoritmo di risoluzione. Cliccando il pulsante, il programma esegue l'algoritmo corrispondente e mostra visivamente il percorso trovato, mostrando nel terminale il costo e il tempo di percorrenza del rispettivo.

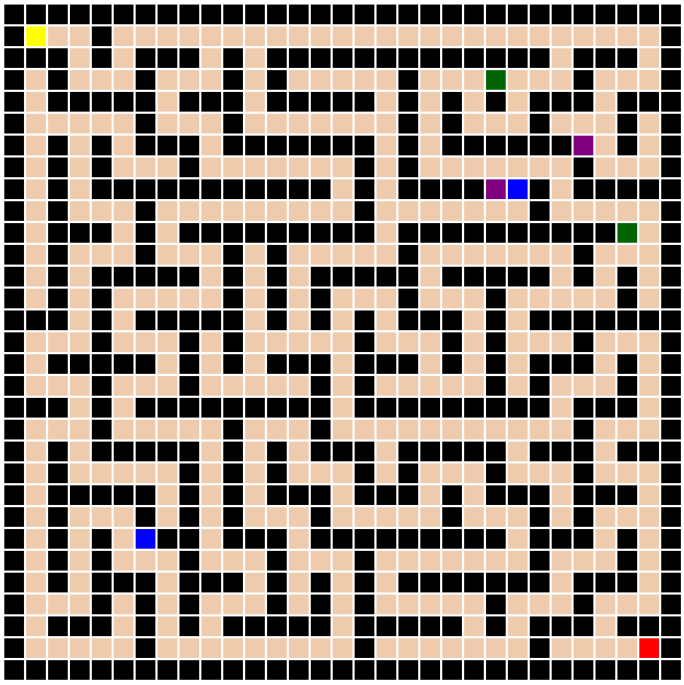
Durante l'esecuzione è possibile visualizzare step by step il progresso della ricerca, ottenendo un confronto visivo dei vari percorsi considerati dall'algoritmo.

Durante lo sviluppo analizzeremo gli algoritmi singolarmente, valutandone le caratteristiche di performance per offrire un confronto più mirato.

Al termine sarà presentato un confronto tra tempi di esecuzione e costi dei percorsi, dati forniti rispettivamente da un cronometro e dal risultato della ricerca.

Dataset

Il dataset si compone di labirinti auto-generati. I test sono stati eseguiti su un labirinto di dimensione 31*31:



Gli Algoritmi Di Ricerca

È presentata una classificazione degli algoritmi secondo il tipo di ricerca adottato. Osserviamo:

1. Algoritmi di ricerca non informata: algoritmi che esplorano lo spazio degli stati senza utilizzare informazioni aggiuntive sull'obiettivo, ovvero senza conoscere quanto uno stato sia vicino alla soluzione.
    a. Ricerca in ampiezza.
    b. Ricerca a costo uniforme.
    c. Ricerca in profondità con variante limitata e approfondimento iterativo.
2. Ricerca informata: utilizzano informazioni euristiche per guidare la ricerca verso l'obiettivo, riducendo il numero di stati esplorati e migliorando l'efficienza.
    a. Greedy.
    b. A*.
3. Ricerca locale: algoritmi che operano su una singola soluzione (stato) e cercano di migliorarla iterativamente attraverso mosse successive, senza esplorare esplicitamente tutto lo spazio degli stati.
    a. Hill climbing.
    b. Simulated annealing.
    c. Genetici.
    d. Local Beam.

Approfondimenti:

1.b Ricerca a costo uniforme:

È una strategia di ricerca non informata, dunque non è prevista una conoscenza su quanto uno stato sia vicino l'obiettivo. La ricerca a costo uniforme è utilizzata quando le azioni hanno costi differenti fra loro, con lo scopo di ricercare la soluzione dal costo minimo. L'analisi viene gestita con l'utilizzo di una funzione di valutazione:

$$g(n)$$

che rappresenta il costo minimo dalla radice dell'albero di ricerca al nodo corrente. Nel nostro caso dall'inizio del labirinto alla cella corrente.

Le caratteristiche si basano sul costo della soluzione ottima, che denotiamo con C*, e sul limite inferiore del costo di ogni azione, che denotiamo con ε. Assumendo le seguenti considerazioni:

-il passaggio per una qualsiasi cella innocua ha un costo di 1;

-il passaggio per la cella n.4, la melma della disperazione, ha un costo di 50;

-il passaggio per la cella n.5, il muro delle illusioni, ha un costo di 10;

-il passaggio per la cella n.6, il frutto della dimenticanza, ha un costo di 15.

Si può definire un costo di percorso variabile, per cui la complessità spaziale e la complessità temporale sono dinamiche, secondo la seguente legge:

$$O(b^{1+\frac{C*}{e}})$$

Osservando il labirinto, si calcola il costo minimo C* pari a 84 passi, senza attraversare alcun ostacolo, quindi:

$$C^* = 84$$

Il valore del limite minimo è proprio:

$$\varepsilon = 1$$

Si definisce b, il fattore di ramificazione, il numero di azioni possibili dal nodo corrente, pari a:

$$b = 3$$

perché Sarah può spostarsi in avanti, indietro, a destra o a sinistra. All'inizio può sembrare abbia a disposizione quattro mosse, ma considerando la presenza del muro, si riduce questo fattore a tre.

Non sono da considerarsi come azioni tutti gli input per rilevare la presenza di ostacoli; quindi, non modificano il fattore di ramificazione, ma influiscono sul costo minimo C*.

Affrontando le considerazioni finali per la valutazione della complessità spaziale, si può calcolare:

$$\text{SPAZIO} = O(3^{1+\frac{84}{1}}) = O(3^{85}) = \text{TEMPO}$$

Valgono le stesse considerazioni per la complessità temporale. Un valore così alto indica che l'esecuzione dei calcoli e l'analisi della ricerca sono di difficile realizzazione in un tempo realistico e ragionevole. Questo perché l'algoritmo esplora molte strade, di cui non tutte portano ad una soluzione utile, sprecando tempo e spazio.

Un algoritmo per essere definito completo deve poter garantire sempre una soluzione se esite. L'algoritmo UCS esplora gli stati in ordine crescente di costo; quindi, non si perderà in percorsi di costi maggiori finché non esplora quelli con costi minori. Infatti, quando l'algoritmo si avvicina all'obiettivo il numero dei percorsi considerati diminuisce. Condizione necessaria è che ogni azione abbia un costo minimo positivo, questo perché esplorando gli stati in ordine crescente di costo, non può considerare costi nulli o negativi. Un modo utile per visualizzare una ricerca consiste nel disegnare i confini nello spazio degli stati. In questo caso si avranno confini circolari che si diffonderanno uniformemente in tutte le direzioni, senza preferenze verso l'obiettivo. Confermando di non essere trattabile in tempi realistici. Concludendo l'analisi di completezza, dato che il costo minimo di ogni azione è strettamente positivo ($\varepsilon=1$) ed il costo minimo C* è finito, allora l'algoritmo in questo scenario è completo.

COMPLETO= SI

Concludendo la valutazione delle prestazioni con l'aspetto ottimo dell'algoritmo UCS, si può dire che:

OTTIMO= SI

Perché, definito il costo di ogni azione strettamente positivo, UCS espande sempre il nodo della frontiera con costo cumulativo più basso, garantendo che la prima soluzione sia quella con costo totale minore, la soluzione ottimale.

Infine, l'algoritmo UCS può essere scelto se si vuole trovare il percorso con costo minimo, considerando il labirinto con costi dinamici. Se si vuole ottimizzare la soluzione in base alla memoria utilizzata, questo algoritmo non è la scelta giusta.

2.b A*:

È un tipo di strategia di ricerca informata, quindi vengono forniti suggerimenti per la ricerca dell'obiettivo sulla base della conoscenza sul dominio applicativo. I suggerimenti hanno la forma di una funzione euristica:

$$h(n)$$

questo tipo di ricerca utilizza una funzione di valutazione,

$$f(n)= g(n) + h(n)$$

dove:

g(n) indica il costo del cammino dal nodo iniziale al nodo n.

h(n) indica il costo stimato del cammino più breve dal nodo n all'obiettivo.

In questo algoritmo si può considerare come euristica la distanza di Manhattan, perché sono ammessi solo spostamenti in direzioni ortogonali, non diagonali. Praticamente è rappresentata dalla somma delle distanze assolute tra le coordinate orizzontali e verticali di due punti.

È un tipo di euristica ammissibile e consistente. Ammissibile perché non sovrastima mai il costo reale per raggiungere l'obiettivo. Consistente perché cresce in modo coerente lungo il cammino, rispettando la disuguaglianza triangolare:

-per ogni nodo n, dato il suo successore n' e l'azione per produrlo a, si ottiene:

$$h(n) \leq c(n, a, n') + h(n')$$

Per valutare l'algoritmo definiamo le sue prestazioni:

la complessità spaziale, coincidente con quella temporale, dipende fortemente dal tipo di euristica considerata. Quando l'euristica è poco informativa, cioè h(n)=0, si ha un comportamento simile all'algoritmo UCS; quindi, è possibile riprendere le considerazioni precedenti.

Tuttavia, quando si utilizza un'euristica ammissibile, A* tende ad esplorare molti meno nodi, concentrandosi in direzione dell'obiettivo. Questa osservazione è visibile con l'estensione dei confini di ricerca. La ricerca A* si espande a partire dal nodo iniziale secondo bande concentriche di f-costo crescente.

Approfondendo un confronto tra UCS e A*, quando si estendono i confini di ricerca, il primo algoritmo estende secondo confini circolari di una dimensione g rappresentante di un solo costo; il secondo, invece, estende confini indirizzandosi sempre più vicino all'obiettivo grazie al contributo della h(n). Questa estensione avviene secondo una disuguaglianza:

$$f(n) \leq f(n')$$

conseguenza della consistenza dell'euristica, quindi il costo del cammino aumenta sempre mentre lo si percorre.

Concludendo l'analisi di questa prestazione, si può dire che questo algoritmo richiede una notevole quantità di memoria perché tiene traccia di tutti i nodi visitata e della frontiera, salvando tutte le alternative da considerare. Risulta, tuttavia, più rapido dell'UCS.

$$\text{SPAZIO} = O(b^{\alpha}) = \text{TEMPO}$$

Con $\alpha < 1 + \frac{C*}{e}$, nel caso ottimo invece diventa:

$$\text{SPAZIO} = O(d) = \text{TEMPO}$$

Dove d è la profondità della soluzione ottima, in questo caso: O(131).

Per quanto riguarda la prestazione ottima dell'algoritmo, dev'essere discussa sulla base dell'ammissibilità dell'euristica.

-se l'euristica è ammissibile, allora la ricerca A* è ottima rispetto il costo.

$$h(n) \leq h^*(n)$$

dove:

-h(n) indica il costo stimato dal nodo n all'obiettivo;

-h*(n) indica il vero costo minimo dal nodo n all'obiettivo.

Validando la teoria che non viene mai sovrastimato il costo; quindi, A* trova sempre il percorso di costo minimo, cioè la soluzione ottimale.

Nel caso in cui venisse applicata un'euristica inammissibile, si sovrastima il costo reale. Questo porta l'algoritmo a trascurare percorsi ottimali perché valuta come troppo costosi i nodi vicini alla soluzione vera e preferisce un percorso apparentemente più economico, ma in realtà più lungo e costoso. Quindi, può funzionare molto più velocemente e trovare una buona soluzione, ma non ottima.

OTTIMO = Si

Combinando la complessità spaziale e l'ottimabilità dell'algoritmo, possiamo dire che A*:

-espande sempre tutti i nodi con $f(n) < C^*$;

-potrebbe espandere la frontiera con $f(n) = C^*$;

-non espande i nodi con $f(n) > C^*$.

Questo permette di effettuare una potatura dell'albero sui nodi non necessari per trovare una soluzione ottima.

La completezza dell'algoritmo è verificata dal rispetto delle seguenti condizioni necessarie:

-lo spazio di ricerca è finito. Necessario perché l'algoritmo espande i nodi finché non trova la soluzione ottimale.

-il costo minimo delle azioni è strettamente positivo, $\varepsilon>1$. Necessario per evitare di entrare in cicli infiniti.

-l'euristica è ammissibile.

Queste condizioni sono rispettate all'interno del labirinto sviluppato, la prima considerando la griglia 31x31 limitata, la seconda considerando il costo di ogni passo e la terza considerando la distanza di Manhattan.

In conclusione, A* offre prestazioni migliori di UCS in presenza di un'euristica ammissibile, mantenendo completezza e ottimalità, ma con un costo in memoria più elevato. UCS, pur garantendo la soluzione ottima, è meno efficiente in problemi complessi. La scelta dell'algoritmo dipende dall'obiettivo: ottimizzazione del costo, efficienza computazionale o disponibilità di conoscenze euristiche.

3.b: Simulated Annealing:

È un algoritmo di ricerca locale, opera a partire da uno stato iniziale e procedendo verso gli stati adiacenti, senza tenere traccia ne dei cammini ne degli stati già raggiunti. Non esplora tutti i cammini, evitando anche quello in cui potrebbe risiedere la soluzione, questo lo rende un algoritmo non sistematico e soprattutto non completo.

COMPLETO = NO

Considerato lo spazio degli stati, ogni stato ha un'altezza definita dal valore della funzione obiettivo:

-se l'altezza corrisponde alla funzione obiettivo, lo scopo è trovare il picco più alto, il massimo globale;

-se l'altezza corrisponde al costo, lo scopo è trovare il punto più basso, il minimo globale.

Il nostro caso corrisponde al secondo, la funzione obiettivo indica il costo che vogliamo minimizzare.

Questo algoritmo è un'estensione dell'hill climbing, che lo rende più generalizzato e migliorato. Il SA, durante l'esplorazione del percorso, accetta anche peggioramenti, con una certa probabilità:

$$p = e^{\frac{-\Delta E}{T}}$$

indichiamo con:

-ΔE, la variazione tra il costo della nuova soluzione e quella attuale. Chiaramente se lo stato successivo ha un costo minore di quello corrente, viene accettato sicuramente;

-T, la temperatura. Parametro astratto per indicare il controllo sulla probabilità di accettare una mossa peggiorativa. In particolare, una T alta ci consente di accettare spesso le soluzioni peggiori, mentre una T bassa predilige le soluzioni migliori e l'algoritmo diventa molto selettivo.

Questo approccio consente all'agente di non restare bloccato in minimi locali, accettando peggioramenti per uscirne.

Analizzando la formula:

-per $T \to \infty$, $p = e^{\frac{-\Delta E}{T}} \approx e^{0} = 1$. Questo vuol dire che nei casi iniziali, quando assumiamo una temperatura alta, l'algoritmo accetta quasi sempre le mosse peggiorative, evitando che sin dall'inizio l'algoritmo resti bloccati in minimi locali.

- per $T \to 0$, $p = e^{\frac{-\Delta E}{T}} \approx e^{-\infty} = 0$. Questo vuol dire che nell'avanzamento dell'analisi, l'algoritmo accetterà sempre di più solo mosse migliorative, cercando di stabilirsi su un buon minimo.

Dunque, se la temperatura scende drasticamente, non è possibile osservare tutti i percorsi disponibili. L'algoritmo diventa presto troppo selettivo, non accettando mosse peggiorative e comportandosi come l'algoritmo hill climbing. Quindi la probabilità di trovare il minimo globale è bassa. Se, invece, adottiamo un 'raffreddamento' lento, l'algoritmo ha più tempo per esplorare varie soluzioni, forse con costo più alto, ma ottenendo un valore per la probabilità di trovare il minimo globale tendente a 1.

Sulla base delle stesse considerazioni, analizziamo la complessità temporale. È chiaro che un lento raffreddamento comporta un intervallo di tempo ampio. Invece, un raffreddamento drastico rende il principio più rapido. Possiamo, dunque, affermare che l'ottimalità e la complessità temporale sono

una il reciproco dell'altra, perché scegliendo un raffreddamento rapido otteniamo un algoritmo non ottimo, ma veloce. Viceversa, con un raffreddamento lento otteniamo un algoritmo ottimo, ma lento.

OTTIMO= DIPENDE

TEMPO= DIPENDE

Come è stato già riportato, SA non tiene traccia ne dei cammini ne degli stati già raggiunti; quindi, non ha bisogno di una grande dimensione proporzionale alla dimensione dello spazio degli stati. Ciò che conserva sono:

-lo stato corrente, per considerare la soluzione corrente;

-stato successivo, per calcolare il nuovo costo e decidere se accettarlo;

-parametri (temperatura, variazione di costo, probabilità).

Dunque, la complessità spaziale è generalmente bassa e proporzionale alla dimensione di uno stato, utile per grandi spazi di ricerca.

SPAZIO= O(s)

1.a Ricerca in ampiezza

Si tratta di un tipo di ricerca non informata che esplora i labirinti in maniera sistematica, iniziando l'espansione dalla posizione di partenza, espandendo i successori, poi i successori di questi ultimi, fino al raggiungimento della soluzione. Per realizzare tale comportamento viene adoperata una coda di tipo FIFO: le nuove celle vanno in fondo alla coda, mentre, quelle più vecchie sono posizionate ad una profondità minore in modo tale da essere espanse per prime. Ogni stato viene raggiunto con il cammino migliore, per cui il test obiettivo è anticipato al momento in cui la cella viene generata. Trattandosi di una ricerca sistemati, questa è completa per labirinti il cui fattore di ramificazione (b) è finito. Se non fosse finito e la soluzione non si trovasse al primo livello, l'esplorazione non procederebbe in profondità bloccando l'algoritmo.

La complessità temporale è esponenziale sulla profondità dell'albero d:

$$1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$$

Data la necessità di memorizzare tutte le informazioni risulta oneroso anche in termini di spazio, anche in questo caso con la complessità esplode all'aumentare della profondità:

$$O(b^{d+1})$$

Nella realtà, i requisiti di memoria rappresentano il problema principale di questo approccio, che non risulta sempre applicabile. Anche i costi temporali, tuttavia, non sono trascurabili.

Può essere sfruttato in situazioni in cui i costi azione sono tutti uguali, allora restituisce le soluzioni con il minor numero di passi. Invece, con costi non uniformi, ignora i costi e considera solo la profondità, non restituendo la soluzione ottimale, ma la prima che viene estratta dalla coda. Ricapitolando:

- È completo.
- Non è sempre ottimo.
- Costo temporale: $O(b^{d+1})$
- Costo spaziale: $O(b^{d+1})$

3.d Local Beam:

Si tratta di un algoritmo di ricerca locale che tiene traccia di k stati anziché uno solo. I k stati iniziali sono generati casualmente e ad ogni passo si verifica se uno qualsiasi dei loro successori abbia raggiunto l'obiettivo finale. Se questa condizione è verificata, l'algoritmo termina, altrimenti si scelgono i k stati migliori della lista e continua (quelli peggiori sono scartati).

Le ricerche avviate sono eseguite in parallelo e le informazioni utili sono scambiate tra una ricerca e l'altra. È questo comportamento che lo distingue da hill climbing e permette di abbandonare le ricerche meno produttive per dirigere le risorse dove si verificano i progressi maggiori.

Questo approccio soffre di una carenza di diversificazione tra i vari stati, che possono concentrarsi in cluster in una piccola regione dello spazio degli stati, rendendo la ricerca una versione simile ad hill climbing se non peggiore.

Andando ad aumentare il valore di k (detto ampiezza del fascio) si escludono meno stati dalla ricerca; infatti, se si potesse impostare ad infinito non si escluderebbe nessuno stato e il comportamento sarebbe identico ad una best-first-search. Trattandosi di una ricerca locale:

- Non è ottima in quanto scarta molti stati potenzialmente promettenti che tuttavia non rientrano nei migliori k. Può anche bloccarsi in un massimo locale ed anche la funzione euristica potrebbe influenzare negativamente l'algoritmo.
- Non è completa dato che non esplora tutto lo spazio delle soluzioni, ma solo i percorsi promettenti per l'euristica. Se tutti i k stati terminassero in un vicolo cieco l'algoritmo terminerebbe senza trovare alcuna soluzione.

Complessità spaziale e temporale dipendono da:

- Fattore di ramificazione.
- Profondità della soluzione (nel caso in cui questa sia raggiunta).
- Numero di cammini k.

Ogni iterazione genera $k * b$ stati mantenendone solo i k migliori, con d livelli da considerare,ne risulta un costo temporale di:

$$O(b * k * d)$$

Il costo a livello di spazio è simile, considerando la necessità di mantenere in memoria solo i k stati

$$O(b * k)$$

Ricapitolando:

- Non è completo.

- Non è ottimo.
- Complessità temprale: $O(b * k * d)$
- Complessità spaziale: $O(b * k)$

3.c Genetici:

si tratta di algoritmi di ricerca locali molto simili alla local beam, che permettono di generare i k stati successori a partire dai migliori precedenti. Presa la popolazione di individui più promettenti si forma la generazione successiva in un processo detto ricombinazione. Alla base del suo funzionamento c'è una stringa che permette di codificare le varie configurazioni. Le nuove generazioni possono anche essere create prendendo, ad esempio, l'inizio di un percorso e la fine di un altro, proprio per questo è importante poter lavorare sulle stringhe anche come sottostringa.

Un tipico algoritmo genetico si compone del seguente schema di funzionamento di base:

1. Generazione casuale della prima popolazione di soluzioni (cromosomi).
2. Applicazione della funzione di valutazione alle soluzioni (cromosomi) appartenenti all'attuale popolazione.
3. Selezione delle soluzioni considerate migliori in base al risultato della funzione di fitness e della logica di selezione scelta.
4. Procedimento di crossover per generare delle soluzioni ibride a partire dalle soluzioni scelte al punto *3*.
5. Creazione di una nuova popolazione a partire dalle soluzioni identificate al punto *4*.
6. Riesecuzione della procedura a partire dal punto *2* ed utilizzando la nuova popolazione creata al punto *5*.

Il cromosoma rappresenterebbe una delle soluzioni al problema considerato. Con crossover si intende il processo di generazione di una nuova soluzione mescolando delle soluzioni esistenti. Si tratta di concetti ereditati dal campo della biologia da cui l'algoritmo trae ispirazione.

Per quanto riguarda le proprietà:

- Non garantendo una soluzione ottima globale non è ottimo. Anche in questo caso può capitare che si blocchi in ottimi locali.
- Non è completo.
- Presenta un costo temporale di: $O(g * p * f)$.
- La complessità spaziale è: $O(p)$

Sia spazio che tempo sono proporzionali a p, ossia, la dimensione della popolazione. Invece g rappresenta il numero di generazioni ed f il costo di valutazione per ciascun individuo. Ad esempio, nella simulazione di esplorazione presentata:

- Population_size (p) di 30.
- Max_generation (g) di 50.

GREEDY SEARCH (Ricerca Greedy) – Ricerca Informata

Ricerca Informata:

Per parlare di *ricerca informata*, dobbiamo introdurre prima che cos'è un'**euristica.** Quest'ultima rappresenta lo spazio delle soluzioni ed è molto complesso e si fa la ricerca della soluzione più appropriata. le strategie informate possono ridurre drasticamente lo spazio di ricerca utilizzando euristiche rendendo risolvibili problemi che computazionalmente sarebbero proibitivi (cioè troppo costosi).

Descrizione sintetica dell'algoritmo:
La *Greedy Search* (*ricerca avida/grassa*) sceglie sempre il nodo che "sembra" più vicino alla soluzione, basandosi esclusivamente sull'**euristica** *h(n)*, ignorando il costo accumulato (ecco perché si dice "avida" o "grassa"). È come se si cercasse sempre la strada apparentemente più promettente passo dopo passo, e come nel caso del labirinto, quella col minor numero di caselle percorse.

- **Completezza:** Non garantita.
  - Se l'albero di ricerca ha cicli o se si scende in un ramo sbagliato, l'algoritmo può perdersi e non raggiungere mai la soluzione.

- **Complessità Temporale**: **O(b^m)**
  - Dove b è il **fattore di diramazione massimo** e m la **profondità massima dello spazio degli stati**. Può essere molto inefficiente nei grafi ampi, perché può esplorare molte strade non necessarie.

- **Complessità Spaziale**: **O(b^m)**
  - Poiché memorizza tutti i nodi aperti in una *frontiera*, l'uso di memoria può essere significativo.

- **Ottimalità**: **Non ottimale.**
  - Può trovare una soluzione sub-ottimale, perché sceglie ciò che sembra "vicino" ma non necessariamente ciò che ha **costo minimo**.

HILL CLIMBING – Ricerca Locale

Ricerca Locale:

Questi algoritmi rappresentano un approccio fondamentale per risolvere problemi di ottimizzazione di intelligenza artificiale. Si basano solo sullo stato corrente, non esplorando necessariamente tutto lo spazio degli stati e sono per questo motivo (non sempre) più veloci delle ricerche sistematiche.

Descrizione sintetica:
È un algoritmo ispirato all'idea di scalare una montagna: si valuta il "vicino" migliore (ossia il

successore con valore più alto o più basso, a seconda del problema), e si prosegue in quella direzione. Tuttavia, guarda solo ai **vicini immediati**, rischiando di rimanere bloccato in **massimi/minimi locali**. Nel nostro caso è stata impostata la funzione che tende a minimizzare, poiché vogliamo che si raggiunga la fine con il minor numero di caselle percorse.

- **Completezza**: **Non garantita.**
  - Se l'algoritmo resta intrappolato in un massimo/minimo locale, senza "salti" possibili, non troverà mai la soluzione globale.

- **Complessità Temporale**: **O(n^2)** nel caso medio.
  - Il tempo dipende dal numero di passi fino a raggiungere un picco. In problemi con molti picchi locali, può richiedere molte iterazioni (oppure bloccarsi presto).

- **Complessità Spaziale**: **O(1)**
  - Utilizza pochissima memoria, poiché non conserva la struttura dell'albero né uno storico dei percorsi. ("**1**" perché si basa sullo stato corrente)

- **Ottimalità**: **Non ottimale.**
  - Non ha alcuna garanzia di trovare la soluzione migliore (ottima), a meno di implementare varianti come *random restarts* o *simulated annealing*.

Problemi dell'H.C.:

1. **Massimi/Minimi Locali:** La ricerca può bloccarsi in soluzioni subottimali.
2. **Pianure:** Sono delle aree piatte dove non è chiaro come procedere.
3. **Creste:** Sequenza di massimi/minimi locali che ostacola il progresso.

Varianti dell'H.C.:

1. **Random-Restart:** Esegue *Hill Climbing* più volte da stati iniziali casuali.
2. **Random Sideways Moves:** Permette di fare mosse laterali per uscire da pianure (spalle).
3. **Stochastic H.C.:** Sceglie i successori casualmente (pesati per valore) tra i migliori. [Soluzione che è stata adottata proprio perchè in situazioni di minimi locali (che secondo l'algoritmo erano uno stato ottimale) ha permesso di uscire da questi stati].

DEPTH-LIMITED SEARCH (DLS) – Ricerca Non Informata

Ricerca non informata:

Si basa esclusivamente sulla struttura del grafo o dell'albero, esplorando i nodi in modo sistematico (in ampiezza, profondità, o con limiti predefiniti).

Descrizione sintetica:
Equivale all'algoritmo DFS (Depth – First - Search) con limite di profondità **l**, cioè un **limite massimo alla profondità** di esplorazione (che in implementazione è stato definito semplicemente come "*limit*"), prevenendo loop infiniti all'interno del labirinto. Si esplora a fondo ogni ramo fino al

limite stabilito. Infatti ogni volta che, in ogni ramo, raggiunge il limite e non ha raggiunto l'obiettivo (casella Fine = 3) e di conseguenza riparte con il ramo successivo.

- **<u>Completezza</u>: Sì, se il limite è sufficientemente grande.**
  - Tuttavia, se il limite è troppo basso, può "tagliare" via la soluzione (*cutoff*), senza mai trovarla.

- **<u>Complessità Temporale</u>: O(b^l)**
  - Dove l è il limite di profondità. È più efficiente della DFS pura quando si impone un buon limite.

- **<u>Complessità Spaziale</u>: O(l)**
  - Molto contenuta: basta memorizzare la pila della chiamata ricorsiva.

- **<u>Ottimalità</u>: Non ottimale.**
  - Trova una soluzione (se esiste), ma non è garantito che sia la migliore in termini di **costo** o **profondità**.

Confronto tra algoritmi

Quando si affronta un problema di ricerca, la scelta dell'algoritmo dipende dall'obiettivo dell'utente: ottenere la soluzione dal costo minimo, raggiungere l'obiettivo nel minor tempo o garantire l'ottimalità. In base al contesto e alle risorse disponibili, ogni algoritmo offre vantaggi e svantaggi.

L'algoritmo UCS è la scelta giusta se si cerca la soluzione dal costo minimo. È ottimo e completo, ma esplora molti nodi senza definire una direzione preferita, rendendolo lento delle analisi e poco efficiente per labirinti dalle grandi dimensioni.

L'algoritmo A* è la scelta giusta se si cerca una soluzione ottima e veloce. È ottimo e completo, a differenza dell'UCS non esplora tutti i nodi, ma si indirizza verso una direzione preferita secondo un'euristica ammissibile e consistente. Questo rende le complessità spaziale e temporale più piccole rispetto a quelle dell'algoritmo precedente.

L'algoritmo SA potrebbe essere la scelta giusta quando l'obiettivo è minimizzare il tempo di esecuzione, perché non esplora l'intero spazio degli stati e può fornire rapidamente una soluzione approssimata. Questo non vuol dire che sia la soluzione ottima. Infatti, osservando le varie soluzioni proposte, l'algoritmo non è la scelta giusta per i nostri labirinti, perché la 'temperatura' scende al di sotto di quella minima fissata e l'algoritmo si ferma senza raggiungere la fine.

Osservando il percorso finale nel labirinto, Sarah non arriva alla destinazione perché la temperatura scende al di sotto di quella minima fissata, quindi si ferma e resta intrappolata.

La Ricerca in Ampiezza (BFS) è completa e trova la soluzione con il minor numero di passi, ma non necessariamente quella dal costo minimo se i costi non sono uniformi. È utile con algoritmi piccoli o con una struttura semplice; tuttavia, l'esplorazione sistematica lo rende poco scalabile.

Gli algoritmi genetici sono adatti per problemi in cui lo spazio delle soluzioni è vasto e mal definito. Non possono garantire che la soluzione sia ottima o completa e i risultati variano in base alla configurazione dei parametri. A causa del processo di creazione delle varie generazioni, che va ad aggiungere un costo temporale fisso all'esplorazione, non compete con altri algoritmi presi in considerazione, in particolare per i labirinti di dimensioni non molto grandi.

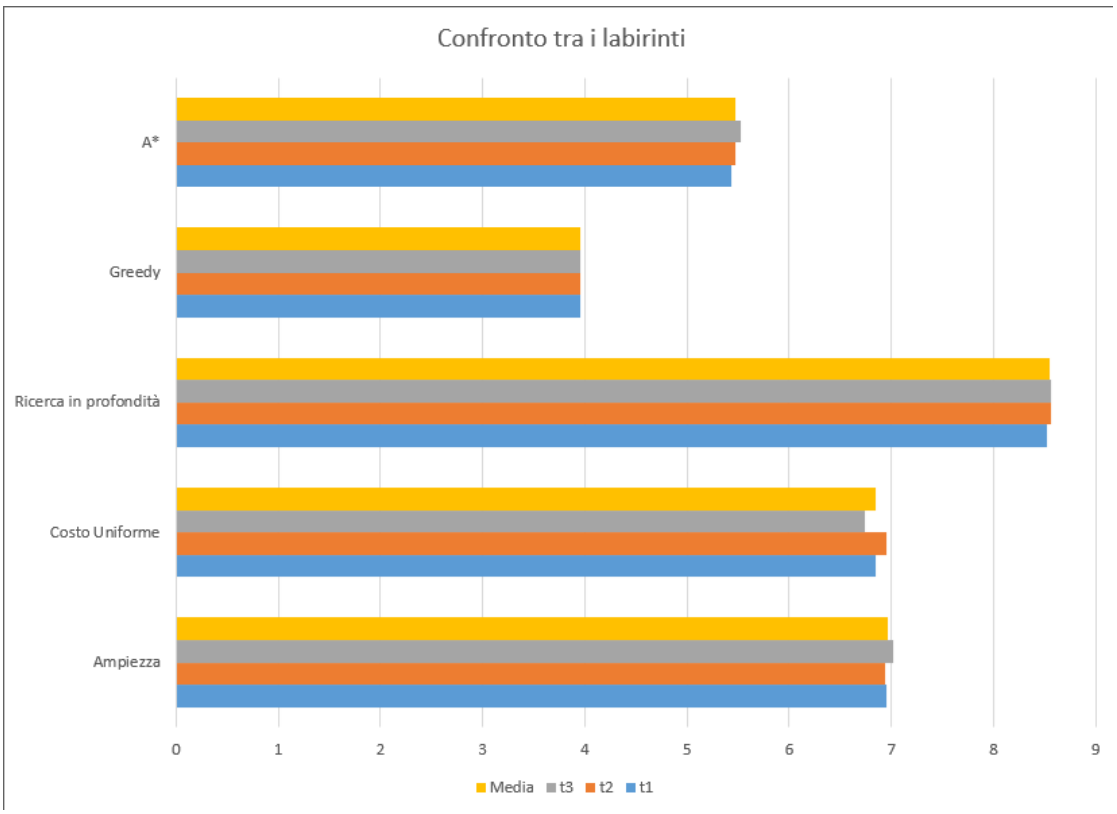
L'algoritmo Local Beam Search è molto veloce, è più robusto rispetto a Hill Climbing, perché sfrutta la diversità iniziale e può evitare di bloccarsi nei massimi locali. Tuttavia, se **i k stati convergono troppo presto alla stessa regione dello spazio**, il comportamento diventa simile a Hill Climbing, con gli stessi limiti. È più veloce e meno dispendioso delle ricerche informate e non-informate, a discapito di completezza e ottimalità.

L'algoritmo Greedy può essere una buona scelta se si desidera trovare una soluzione in tempi brevi, privilegiando i percorsi che appaiono più promettenti sulla base di un'euristica. Tuttavia, non garantisce né completezza né ottimalità: può facilmente scegliere strade apparentemente vantaggiose che poi si rivelano vicoli ciechi. È adatto a problemi in cui si dispone di una buona funzione euristica, ma rischia di fallire in spazi di ricerca complessi.

L'algoritmo Hill Climbing è adatto se l'obiettivo è ridurre al minimo l'uso di memoria, poiché mantiene in considerazione solo il nodo corrente e i suoi vicini. Tuttavia, essendo una tecnica di ricerca locale, tende a fermarsi su minimi locali senza esplorare soluzioni potenzialmente migliori altrove. È veloce e semplice, ma non è né completo né ottimale. Inoltre, la mancanza di memoria storica lo rende inefficace in spazi ricchi di pianure o minimi locali multipli (creste).

L’**algoritmo Depth-Limited Search** (**DLS**) è più indicato quando si ha un controllo preciso sulla profondità della soluzione attesa. Limita la ricerca fino a una certa profondità, evitando così esplorazioni infinite in spazi ciclici. È completo solo se la soluzione si trova entro il limite fissato e non è ottimale, ma consuma poca memoria ed è utile per problemi in cui si può stimare un limite ragionevole; quindi, può essere facilmente associabile ai labirinti. Se il limite è troppo basso, scatta il *cutoff* e taglia la soluzione.

Eseguendo un confronto tra i vari algoritmi (labirinto dalla dimensione 31*31), si osserva che gli unici ad arrivare sempre a destinazione sono: ricerca in ampiezza, costo uniforme, ricerca in profondità, Greedy ed A*.



Nota: t1, t2 e t3 sono tre misurazioni di cui è stata fatta la media.